Analysis and Evaluation of Congestion-Avoidance Mechanisms (Tahoe, Reno, CUBIC) with a Proposed Lightweight Predictive Congestion Estimator

Antra- 24BDS0066
Varshini Harikrishna- 24BCE0780
Anuska Sen- 24BCE2219

PROBLEM AND MOTIVATION

The Problem: Reactive Congestion Control

- Traditional TCP variants such as Tahoe, Reno and CUBIC primarily react to congestion after packet loss occurs. By then, router queues may be full, increasing delay, retransmissions and packet drops.

Why Existing Approaches Fall Short :

- Late response: Congestion is detected after network performance has already degradead.
- Unpredictable competition: Delay-based approaches can lose bandwidth when competing with more aggressive, loss-based senders.
- Limited anticipation: Early signals such as rising RTT are not fully used by traditional loss-based TCP.

Our Motivation: Predict Before Loss

- Predictive TCP uses a lightweight machine-learning decision tree to identify congestion approximately 0.5 seconds ahead and reduce its sending rate proactively, using sender-side measurements.
- Key Finding: Predictive TCP reduces packet loss and delay compared with Reno, but achieves only 40% bandwidth share against competing CUBIC flows, highlighting a trade-off between efficiency and fairness.
- Objective: Reduce congestion-related losses and delay while maintaining throughput and improving fairness.

EXISTING APPROACHES: TCP CONGESTION CONTROL

Algorithm	How it behaves	Main weakness
TCP Tahoe	Increases its window, but after a loss it resets the congestion window to one segment and starts recovering.	Recovery can be slow.
TCP Reno	Increases its window and approximately halves it when congestion is detected through loss.	Often reacts after packets have already been dropped.
TCP CUBIC	Uses a cubic growth function to increase its congestion window.	Can compete aggressively for bandwidth and keep queues full.
Predictive TCP	Uses an ML decision tree to predict future congestion, while retaining Reno's loss response.	Can sacrifice bandwidth fairness when competing with aggressive TCP variants.

Where Existing Approaches Fall Short

1. Reactive congestion handling
   - Tahoe, Reno and CUBIC primarily use packet loss as a congestion signal.
   - By the time loss occurs, queues may already be full, increasing delay and retransmissions.
   - Repeated window reduction and growth can cause unstable throughput and persistent queueing delay.

2. Different algorithms, different trade-offs
   - Approach Observed weakness
   - Loss-based TCP Late response to congestion
   - Delay-based TCP (Vegas) May surrender bandwidth to aggressive senders
   - Model-based TCP (BBR v1) Can overload shallow buffers in certain scenarios

Key gap: Existing approaches do not consistently combine early congestion prediction, low delay and competitive bandwidth sharing.

RESEARCH GAP: PREDICT BEFORE PACKETS ARE LOST

WHAT THE BASELINE COMPARISON REVEALS

Reactive

Loss-based TCP

Detects congestion after packet loss signals a problem.

Predictive

Proposed approach

Uses sender-side measurements to forecast congestion approximately 0.5 seconds ahead.

Observation	Implication
RTT rises as the bottleneck queue fills	Early warning is possible before packet loss
Vegas can lose bandwidth to aggressive flows	Early backoff alone does not guarantee fairness
BBR v1 can overload shallow queues in the tested setup	High throughput alone does not guarantee low loss

PROPOSED NOVELTY

Our project introduces Predictive TCP, a congestion-control approach that attempts to identify congestion before packet loss occurs. Instead of relying only on losses as a warning, it uses a lightweight machine-learning model to anticipate network conditions and adjust the sending rate proactively.

What makes our approach different?

- Early prediction: Forecasts congestion approximately 0.5 seconds ahead using the sender's RTT measurements.
- Lightweight intelligence: Uses a small, explainable decision tree that can be implemented directly in C++.
- Adaptive response: Adjusts TCP's congestion window according to predicted congestion while retaining Reno's normal loss-recovery mechanism.
- The goal is to reduce unnecessary packet drops and queueing delay without significantly sacrificing throughput.

BEYOND PREDICTION: ADAPTIVE CONGESTION CONTROL

Predictive TCP does more than identify congestion. It uses the prediction to decide how the sender should behave.

How the mechanism works

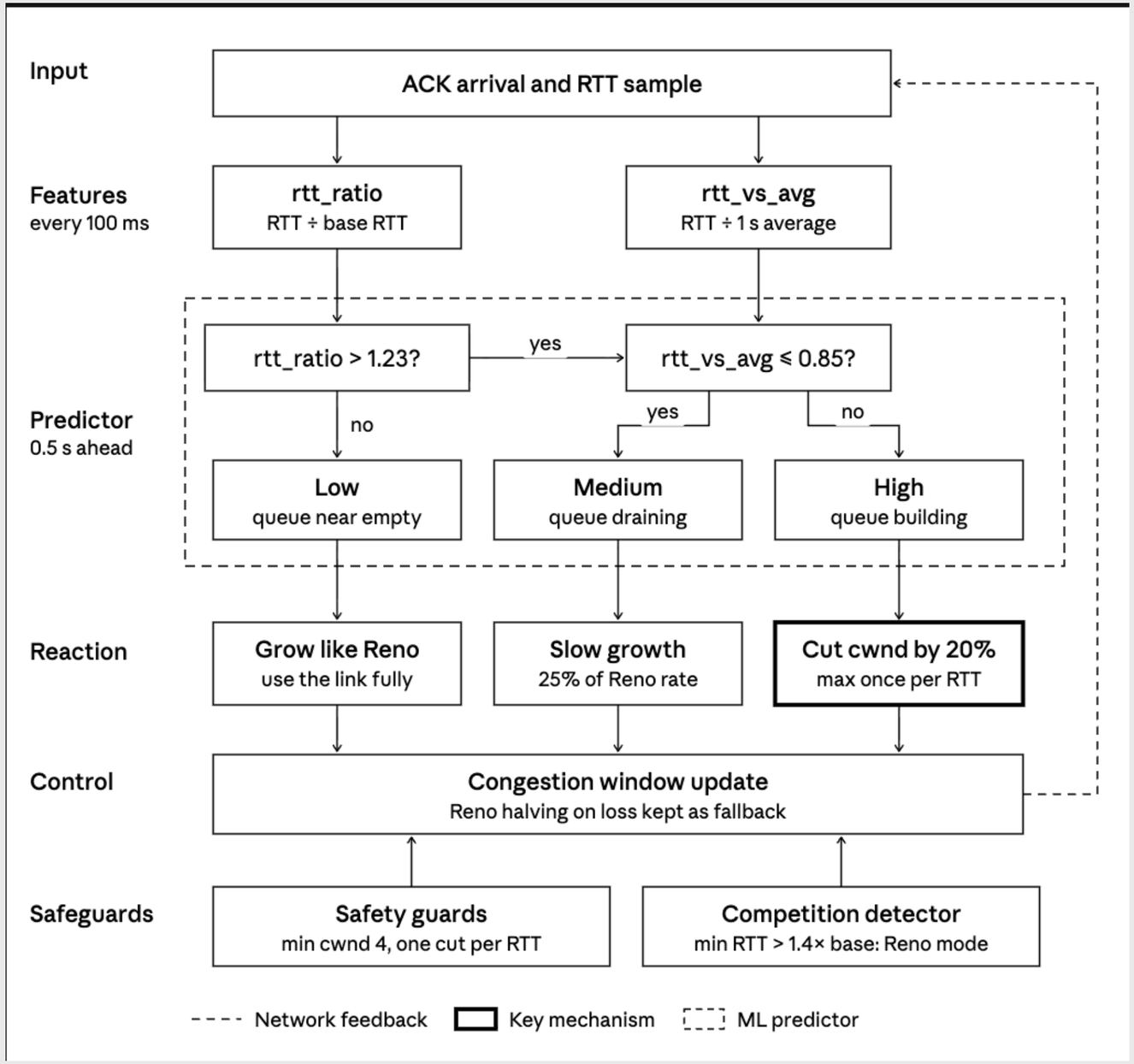
- Monitor: Measure round-trip time (RTT) and track its change over time.
- Predict: Run the decision tree every 100 ms to classify congestion approximately 0.5 seconds ahead.
- Respond: Grow normally under low congestion, grow cautiously under medium congestion, or reduce the congestion window by 20% under high congestion.
- Adapt to competition: A separate detector can switch the sender toward Reno behaviour when persistent queueing suggests aggressive competing flows.

Why it matters

In the tested simulations, Predictive TCP achieved approximately 90% fewer packet drops and 28–36% lower delay than Reno, while retaining 97–100% of Reno's throughput.

Known limitation: It still struggles to obtain a fair bandwidth share when competing with aggressive CUBIC flows.

ARCHITECTURE



ALGORITHM

1. Every ACK gives the sender an RTT sample.
2. Every 100 ms it computes two features: rtt_ratio (RTT ÷ base RTT) and rtt_vs_avg (RTT ÷ its 1 s average).
3. The 4-leaf decision tree predicts congestion 0.5 s ahead. If rtt_ratio ≤ 1.23 the prediction is Low. Otherwise, if rtt_vs_avg ≤ 0.85 it is Medium (queue draining), and if not it is High.
4. Low grows like Reno, Medium grows at 25% of Reno's rate, and High cuts the window by 20% ($\beta = 0.8$), at most once per RTT. The thick box marks this cut because the ablation showed it is the main mechanism.
5. The safety guards and the competition detector gate the window update, and Reno's halving on loss stays as a fallback.
6. The dashed arrow shows the feedback loop: a smaller window gives a shorter queue and a lower RTT, so the next prediction sees the effect.

IMPLEMENTATION SETUP

Simulation Environment

- NS-3.48 network simulator used to model TCP congestion and packet transmission.
- C++ used to implement TCP variants and the proposed Predictive TCP controller.
- Python used for dataset preparation, machine learning, analysis and visualisation.

Dataset and Model

- Generated a dataset from simulated TCP traffic rather than relying on an external dataset.
- Collected RTT, congestion-window and packet-loss measurements at 100 ms intervals.
- Used router queue conditions 0.5 seconds later as prediction labels.
- Trained a lightweight Decision Tree classifier using scikit-learn.

Experimental Setup

- Compared Tahoe, Reno, CUBIC, Vegas, BBR and Predictive TCP.
- Evaluated performance under different congestion levels and buffer sizes.
- Used separate simulation seeds for model tuning and testing.

IMPLEMENTATION SETUP

Parameter	Value	Simple meaning
Bottleneck bandwidth	10 Mbps	Maximum capacity of the slowest link
Bottleneck delay	20 ms	One-way propagation delay of that link
Queue capacity	50 packets	Maximum packets waiting in the queue
Base RTT	44 ms	Round-trip time when the queue is empty
Simulation duration	60 seconds	Duration of each simulated experiment
Random seeds	10	Ten repeatable traffic variations

SCENARIOS & METRICS

The project evaluates the proposed Predictive TCP congestion-control mechanism against five existing algorithms: Tahoe, Reno, CUBIC, Vegas, and BBR. Experiments use a dumbbell network with a 10 Mbps bottleneck link, a 20 ms bottleneck delay, a 50-packet FIFO queue, and a base RTT of 44 ms.
Each simulation runs for 60 seconds and is repeated using 10 random seeds. Seeds 1–3 are used for training and tuning, while seeds 4–10 remain unseen and are reserved for testing generalization.

Low load	3 on/off TCP flows, about 30% load	No congestion: does prediction cost anything?
Medium load	6 on/off TCP flows, about 90% load, bursty	Main test: bursty congestion
High load	8 long-lived TCP flows + 2 Mbps on/off UDP	Main test: saturated link
Single flow	1 flow alone on the link	Can it still fill an empty link?
Buffer size	20- and 100-packet queue instead of 50	Robustness to the buffer size
Share with CUBIC	4 flows of one variant + 4 CUBIC flows, high load	Fairness against loss-based TCP
Link speed and delay	5, 10, 50 Mbps × 20, 100 ms	Robustness to other networks

METRIC	WHAT IT MEASURES	BETTER
Goodput (Mbps)	Useful data delivered per second, without retransmissions	Higher
Average RTT (ms)	Round-trip delay a packet sees, including time waiting in the queue	Lower
Drops	Packets discarded because the bottleneck queue was full	Lower
Retransmissions	Packets the senders had to send again	Lower
Queue (packets)	Average length of the bottleneck queue	Lower
Jain fairness index	How equally the flows share the link (1 = perfectly equal)	Closer to 1
Share vs CUBIC	Share of the goodput when sharing the link with CUBIC (fair = 50%)	Closer to 50%
Prediction quality	Macro-F1 and recall of “High” of the decision tree, on unseen seeds	Higher

RESULTS (BASELINE VS PROPOSED)

RESULTS (BASELINE VS PROPOSED) · MEAN OVER 10 SEEDS

METRIC	RENO · MEDIUM	PREDICTIVE · MEDIUM	CHANGE	RENO · HIGH	PREDICTIVE · HIGH	CHANGE
Goodput (Mbps)	8.49	8.27	−2.6%	8.66	8.62	−0.4%
Average RTT (ms)	81.5	58.7	**−27.9%**	101.5	65.0	**−36.0%**
Drops	660	46	**−93.0%**	1602	164	**−89.8%**
Retransmissions	663	46	**−93.0%**	1413	160	**−88.6%**
Queue (packets)	24.3	6.4	**−73.6%**	43.3	15.2	**−64.9%**
Jain fairness index	0.986	0.986	+0.0%	0.995	0.953	−4.2%

PROPOSED COMPARED WITH	DROPS · MEDIUM	RTT · MEDIUM	DROPS · HIGH	RTT · HIGH
vs Tahoe	**−92% · 10/10 seeds**	**−28% · 10/10 seeds**	**−89% · 10/10 seeds**	**−33% · 10/10 seeds**
vs Reno	**−93% · 10/10 seeds**	**−28% · 10/10 seeds**	**−90% · 10/10 seeds**	**−36% · 10/10 seeds**
vs CUBIC	**−92% · 10/10 seeds**	**−31% · 10/10 seeds**	**−88% · 10/10 seeds**	**−40% · 10/10 seeds**
vs Vegas	+974% · 0/10 seeds	**−9% · 10/10 seeds**	**−36% · 7/10 seeds**	**−39% · 10/10 seeds**
vs BBR	**−98% · 10/10 seeds**	**−34% · 10/10 seeds**	**−98% · 10/10 seeds**	**−40% · 10/10 seeds**

Change = Predictive TCP relative to the baseline; green is better, blue is worse. Every drop and RTT gain over Reno and CUBIC is significant (paired t-test, $p < 0.0001$). Success criteria met: 9 of 9.

ANALYSIS OF RESULTS

Predictive TCP was evaluated against established TCP variants under different simulated congestion conditions.

Key Findings

Reduced packet loss: Approximately 90% fewer packet drops than TCP Reno.

Lower latency: RTT reduced by 28–36%, indicating less queueing delay.

Throughput maintained: Achieved 97–100% of Reno's throughput in the tested scenarios.

Competitive performance: Performed well against several baseline variants, although Vegas remained a strong competitor in some scenarios.

What the Results Tell Us

Using RTT measurements to predict congestion allows the sender to reduce its sending rate before the router queue overflows. However, improvements in packet loss and delay do not automatically guarantee fair bandwidth sharing.

LIMITATIONS

1. Bandwidth fairness

When sharing a bottleneck with CUBIC, Predictive TCP obtains approximately 40% of the bandwidth with the competition detector enabled, compared with 23% without it. The ideal share in the tested 4-vs-4-flow scenario is 50%.

2. Competition-detector false alarms

With a 100-packet buffer at high load, the detector falsely switches to Reno mode for 82–92% of the time in eight of ten seeds. This can reduce the benefit of predictive control.

3. Limited generalisation

Buffer sizes, link speeds and delays were varied, but fairness falls on fast or long links, and the number of competing flows was not varied.

4. Simulation-only evaluation

Results are obtained using an NS-3 dumbbell topology with a FIFO bottleneck queue. Real-world networks may behave differently.

5. Model limitations

Offline macro-F1 is 0.60, below the 0.70 target. The Medium congestion class is particularly difficult to predict, although the High-class recall is 0.86 and meets the project's target.

CONCLUSION

- This project explored whether machine learning can help TCP identify congestion before packet loss occurs.
- Developed Predictive TCP, integrating a lightweight decision tree with Reno-based congestion control.
- Achieved approximately 90% fewer packet drops and 28–36% lower RTT than Reno in the tested simulations.
- Maintained 97–100% of Reno's throughput, demonstrating the potential of proactive congestion management.
- Improved bandwidth share against CUBIC from 23% to 40% using the competition detector, while identifying fairness and false alarms as remaining challenges.
- Final takeaway: Predicting congestion before packet loss can improve network performance, but a practical solution must balance low delay, throughput and fairness.

FUTURE SCOPE

- Evaluate Active Queue Management methods such as FQ-CoDel.
- Explore graded congestion responses instead of switching abruptly between predictive control and Reno behaviour.
- Investigate ProbeRTT-style queue-draining checks to distinguish genuine competing flows from false alarms.
- Replace fixed RTT thresholds with adaptive thresholds that account for changing queue conditions.
- Fix fairness on fast or long links and test different numbers of competing flows.
- Evaluate performance under additional traffic mixes and random seeds.
- Validate the controller using a practical network testbed.
- Study its overhead, stability and fairness under real network conditions.
- Long-term objective: Build a more robust congestion controller that combines early prediction with fair bandwidth sharing across diverse network environments.

REFERENCES

[1] V. Jacobson, "Congestion avoidance and control," in Proc. ACM SIGCOMM, 1988.

[2] K. Fall and S. Floyd, "Simulation-based comparisons of Tahoe, Reno and SACK TCP," ACM Comput. Commun. Rev., 1996.

[3] L. S. Brakmo and L. L. Peterson, "TCP Vegas: End to end congestion avoidance on a global Internet," IEEE J. Sel. Areas Commun., 1995.

[4] J. Mo, R. La, V. Anantharam and J. Walrand, "Analysis and comparison of TCP Reno and Vegas," in Proc. IEEE INFOCOM, 1999.

[5] S. Ha, I. Rhee and L. Xu, "CUBIC: A new TCP-friendly high-speed TCP variant," ACM SIGOPS Oper. Syst. Rev., 2008.

[6] N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh and V. Jacobson, "BBR: Congestion-based congestion control," ACM Queue, 2016.

[7] M. Hock, R. Bless and M. Zitterbart, "Experimental evaluation of BBR congestion control," in Proc. IEEE ICNP, 2017.

[8] V. Arun and H. Balakrishnan, "Copa: Practical delay-based congestion control for the Internet," in Proc. USENIX NSDI, 2018.

[9] K. Tan, J. Song, Q. Zhang and M. Sridharan, "A Compound TCP approach for high-speed and long distance networks," in Proc. IEEE INFOCOM, 2006.

[10] R. Jain, D. Chiu and W. Hawe, "A quantitative measure of fairness and discrimination for resource allocation in shared computer systems," DEC Res. Rep. TR-301, 1984.

[11] M. Allman, V. Paxson and E. Blanton, "TCP congestion control," IETF RFC 5681, 2009.

[12] V. Paxson, M. Allman, J. Chu and M. Sargent, "Computing TCP's retransmission timer," IETF RFC 6298, 2011.

[13] L. Xu, S. Ha, I. Rhee, V. Goel and L. Eggert, "CUBIC for fast and long-distance networks," IETF RFC 9438, 2023.

[14] The ns-3 Project, "ns-3 network simulator." [Online]. Available: https://www.nsnam.org/

[15] S. Abbasloo, C.-Y. Yen and H. J. Chao, "Classic meets modern: A pragmatic learning-based congestion control for the Internet," in Proc. ACM SIGCOMM, 2020.

[16] C.-Y. Yen, S. Abbasloo and H. J. Chao, "Computers can learn from the heuristic designs and master Internet congestion control," in Proc. ACM SIGCOMM, 2023.

[17] F. B. Sarpkaya et al., "BBR's sharing behavior with CUBIC and Reno," arXiv:2505.07741, May 2025.

[18] "Promises and potential of BBRv3," in Proc. Passive and Active Measurement Conf. (PAM), 2024. [Online]. Available: https://inet-bbrv3eval.mpi-inf.mpg.de/

THANK YOU